UNIVERSIDAD MAYOR DE SAN ANDRÉS

Facultad de Ingeniería
Ingeniería Electrónica



ETN 902 - SISTEMAS DE CONTROL II
ETN 1034 - APLICACIÓN DE TÉCNICAS DE CONTROL

LABORATORIO NRO. 5

IMPLEMENTACIÓN DE SOFTWARE EN MPLAB XC8 PARA EL CONTROL DEL BRAZO ROBÓTICO DE 6 GDL

Presentado por:
Bismark CHOQUE CHOQUE
Lucas Daniel CONDARCO CALLE
Rodrigo Miguel IPORRE COCA

Docente:
Ing. Jorge Antonio NAVA AMADOR

Auxiliares:
Univ. Fabian Manuel PLATA VARGAS
Univ. Limber AJNOTA CAHUAYA

La Paz - Bolivia

2026

Índice

1. Resumen **4**

2. Objetivos **4**
2.1. Objetivo General 4
2.2. Objetivos Específicos 4
2.3. Alcance 5

3. Marco Teórico **5**
3.1. Microcontrolador PIC16F1946 5
3.2. Entorno MPLAB X IDE y Compilador XC8 5
3.3. Palabra de Configuración (Configuration Word) 6
3.4. Control de Motores Paso a Paso en Lazo Abierto 6
3.5. Anti-rebote por Software 6
3.6. Interfaz ICSP y Programación In-Circuit 6
3.7. Relación con la Guía del Laboratorio 5 7

4. Descripción del Hardware y Asignación de Pines **7**
4.1. Mapa de Entradas — Pulsadores de Control 7
4.2. Mapa de Entradas — Sensores de Fin de Carrera 8
4.3. Mapa de Salidas — Señales a Drivers de Motores 8

5. Configuración del Entorno MPLAB X IDE **8**
5.1. Instalación y Creación del Proyecto 8
5.2. Configuración del Compilador XC8 9
5.3. Compilar y Verificar 9

6. Estructura General del Programa y Diagrama de Flujo **9**
6.1. Arquitectura: Super-Loop con Máquina de Estados 9
6.2. Diagrama de Flujo Completo del Algoritmo 10
6.3. Variables Globales y su Propósito 12

7. Funciones del Programa — Análisis Detallado **12**
7.1. Función `delay_ms()` — Retardo en Milisegundos 12
7.2. Función `delay()` — Retardo en Segundos 12
7.3. Función `IO_init()` — Inicialización de Puertos 13
7.4. Función `PWM_init()` — Inicialización del Módulo PWM 14
7.5. Función `check()` — Lectura de Sensores de Home 14
7.6. Función `keycan()` — Lectura de Pulsadores de Control 14

8. Modo Manual — Función `hand()` **15**
8.1. Lógica General del Modo Manual 15
8.2. Código del Modo Manual — Eje 1: Base 16
8.3. Código del Modo Manual — Eje 2: Hombro 17
8.4. Código del Modo Manual — Eje 3: Codo 18
8.5. Código del Modo Manual — Eje 4: Muñeca 19
8.6. Código del Modo Manual — Eje 5: Pinza (Abrir/Cerrar) 20
8.7. Código del Modo Manual — Eje 6: Giro de Pinza 21

8.8. Resumen de Ejes y Señales del Modo Manual 22

9. Modo Automático — Las Cinco Secuencias 22
9.1. Arquitectura del Modo Automático 23
9.2. Fundamento de los Tiempos de Delay 23
9.3. Secuencia 1 — Posicionamiento Inicial 24
9.4. Secuencia 2 — Movimientos Secundarios 25
9.5. Secuencia 3 — Movimientos Combinados 25
9.6. Secuencia 4 — Movimientos de Base y Pinza 26
9.7. Secuencia 5 — Movimientos Combinados Finales 26
9.8. Resumen de las 5 Secuencias 27
9.9. Relación con el Modelo Cinemático 27
9.10. Limitaciones del Modo Automático 28

10. Rutina de Homing — Función `rest()` 28

11. Proceso de Grabación del Firmware al PIC 30
11.1. Materiales Necesarios 30
11.2. Conexión del PICkit 3 al Hardware 30
11.3. Procedimiento de Grabación 30
11.4. Verificación del Funcionamiento 31

12. Relación del Código con el Modelo Cinemático 31

13. Resultados y Discusión 31
13.1. Resultados Obtenidos 31
13.2. Implementación del Algoritmo según la Guía del Lab 5 32
13.3. Limitaciones Identificadas 32

14. Conclusiones 33

15. Recomendaciones 33

16. Referencias 34

1. Resumen

El presente informe documenta el proceso completo de implementación del software de control (firmware) para el Brazo Robótico de 6 Grados de Libertad (GDL) modelo XK-LS5B, desarrollado en lenguaje C con el entorno **MPLAB X IDE** y el compilador **XC8** de Microchip Technology, para el microcontrolador **PIC16F1946**.

Se describe en detalle cada etapa del proceso: la configuración del entorno de desarrollo, la asignación de puertos de entrada/salida, la inicialización del módulo PWM, la lógica de anti-rebote por software para pulsadores, el **modo de control manual** (pulsador a pulsador), el **modo automático** (cinco secuencias preprogramadas), la rutina de *homing* o retorno a posición de referencia, y el procedimiento de carga del firmware al hardware real mediante el programador PICkit 3.

En concordancia con los lineamientos del Laboratorio 5 (implementación de algoritmos de control en elementos de procesamiento), se justifica técnicamente cada decisión de diseño, se presenta el diagrama de flujo completo del algoritmo, y se relaciona el código fuente con los modelos cinemáticos obtenidos en los Laboratorios 1 al 4. El resultado es un sistema funcional capaz de ejecutar movimientos controlados en modo manual y secuencias automatizadas de demostración, con detención de emergencia inmediata y rutina de calibración por sensores de efecto Hall.

2. Objetivos

2.1. Objetivo General

Implementar, documentar y validar el software de control completo para el brazo robótico de 6 GDL en el microcontrolador PIC16F1946, utilizando el entorno MPLAB X IDE con el compilador XC8, explicando cada módulo del programa, el flujo del algoritmo implementado y el procedimiento de carga al hardware real.

2.2. Objetivos Específicos

- Configurar el entorno MPLAB X IDE y el compilador XC8 para el PIC16F1946.
- Describir la arquitectura del programa en forma de máquina de estados (super-loop) y presentar su diagrama de flujo completo.
- Explicar la lógica del modo manual con anti-rebote por software de 20 ms.
- Documentar las cinco secuencias del modo automático y su relación con la cinemática directa del brazo.
- Detallar la rutina de *homing* y el uso de los sensores de fin de carrera NJK-5002C.
- Analizar las limitaciones del control en lazo abierto implementado y proponer mejoras hacia un control en lazo cerrado con encoders.
- Describir el procedimiento de grabación del firmware al PIC mediante PICkit 3 e interfaz ICSP.

2.3. Alcance

Este laboratorio cubre la implementación en software (firmware) del sistema de control del brazo robótico. El programa opera en **lazo abierto** para el posicionamiento de los motores paso a paso, utilizando los sensores únicamente como protección de fin de carrera y referencia de posición inicial (homing). No se incluye el rediseño del hardware ni el modelado dinámico completo.

3. Marco Teórico

3.1. Microcontrolador PIC16F1946

El PIC16F1946 es un microcontrolador de 8 bits fabricado por Microchip Technology, perteneciente a la familia *Enhanced Mid-range PIC16*. Sus características principales relevantes para este proyecto se resumen en la Tabla 1.

Cuadro 1: Características del PIC16F1946

Característica	**Valor**
Arquitectura	RISC 8 bits
Memoria Flash	14 KB (8192 palabras de 14 bits)
RAM	1024 bytes
EEPROM	256 bytes
Velocidad CPU	8 MIPS (a 32 MHz)
Pines de E/S	54
Conversor ADC	10 bits, 17 canales
Módulos CCP/PWM	2 (CCP1 y CCP2)
Comunicaciones	UART, I^2C, SPI
Voltaje de operación	1.8 V – 5.5 V
Oscilador interno	Hasta 32 MHz

En este proyecto, el PIC opera a **8 MHz** con oscilador HS externo (cristal de cuarzo), entregando 1 MIPS de velocidad de procesamiento efectiva, suficiente para el control de los 6 ejes mediante retardos (delays) por software.

3.2. Entorno MPLAB X IDE y Compilador XC8

MPLAB X IDE es el entorno de desarrollo integrado oficial de Microchip para todos sus microcontroladores. Es multiplataforma (Windows, Linux, macOS) y se integra de manera nativa con el compilador XC8.

XC8 es el compilador de C estándar (C90/C99) para microcontroladores PIC de 8 bits. La cadena de herramientas completa es:

$$\texttt{código fuente (.c)} \xrightarrow{\text{XC8}} \texttt{objeto (.o)} \xrightarrow{\text{Linker}} \texttt{firmware (.hex)} \xrightarrow{\text{PICkit 3}} \texttt{PIC16F1946}$$

3.3. Palabra de Configuración (Configuration Word)

La palabra de configuración (`__CONFIG`) programa los *fuses* del PIC, bits no volátiles que determinan el comportamiento fundamental del dispositivo antes de ejecutar cualquier instrucción. En el programa se utiliza:

```
__CONFIG(0x3CE1);   // Fuses: HS, WDT off, PWRT on, MCLR on
#define _XTAL_FREQ 8000000  // Cristal de 8 MHz
```

Listing 1: Palabra de configuración del PIC16F1946

El valor `0x3CE1` configura: oscilador HS (High Speed Crystal para 8 MHz), Watchdog Timer deshabilitado, Power-up Timer habilitado, MCLR habilitado y protección de código deshabilitada.

3.4. Control de Motores Paso a Paso en Lazo Abierto

Los motores NEMA 23 y NEMA 17 utilizados en el brazo son motores paso a paso bipolares. En lazo abierto, el control se realiza enviando una señal lógica a los drivers (2M542-N y 2MA320), que internamente generan la secuencia de corriente en las bobinas. La posición angular se controla indirectamente mediante el **tiempo de activación** de la señal:

$$\theta_i = k_i \cdot t_i \quad \text{[grados]} \tag{1}$$

donde k_i es la constante de velocidad angular del motor i (grados/segundo) y t_i es el tiempo de activación en segundos. Esta relación es la base de todos los movimientos del programa.

3.5. Anti-rebote por Software

Los pulsadores mecánicos generan múltiples transiciones eléctricas (*bouncing*) durante los primeros 5–20 ms tras ser accionados. Si no se filtra este efecto, el microcontrolador interpreta un solo pulsado como múltiples eventos. La solución implementada es el **anti-rebote por software**: al detectar un flanco de bajada (pulsador presionado), se espera 20 ms y se verifica nuevamente el estado del pin. Solo si sigue en nivel bajo se considera una pulsación válida.

3.6. Interfaz ICSP y Programación In-Circuit

La interfaz **ICSP** (*In-Circuit Serial Programming*) permite grabar el firmware en el PIC sin retirarlo del circuito. Utiliza 5 señales: V_{PP}, V_{DD}, GND, PGD (datos) y PGC (reloj). El programador PICkit 3 genera estas señales desde la PC mediante USB y el software MPLAB X IDE.

3.7. Relación con la Guía del Laboratorio 5

La guía del Laboratorio 5 establece que se debe implementar un algoritmo de control en un elemento de procesamiento real, configurando temporizadores, entradas/salidas y verificando el desempeño sobre la planta. En este proyecto:

- El **elemento de procesamiento** es el PIC16F1946 (microcontrolador de 8 bits).
- El **algoritmo de control** es el super-loop con máquina de estados (manual/automático) implementado en C con XC8.
- Los **temporizadores** se implementan mediante retardos por software (`delay_ms()` y `delay()`) y el módulo PWM basado en Timer2/Timer4.
- Las **entradas** son los pulsadores del panel y los sensores de efecto Hall; las **salidas** son las señales a los drivers de los motores.
- La **saturación** de la acción de control se implementa mediante las banderas de límite (`zxxw`, `yxxw`, etc.) que impiden activar un motor si ya se alcanzó el tope mecánico.

4. Descripción del Hardware y Asignación de Pines

4.1. Mapa de Entradas — Pulsadores de Control

Los pulsadores del panel están conectados con resistencias pull-up (nivel lógico en reposo: **alto = 1**; al presionar: **bajo = 0**).

Cuadro 2: Pines de entrada — Pulsadores de control

Macro	Pin	Función	Acción al presionar
`kzx`	RA0	Base izquierda	Gira base izquierda
`kyx`	RA1	Base derecha	Gira base derecha
`kss1`	RA2	Hombro arriba	Eleva brazo grande
`kxj1`	RA3	Hombro abajo	Desciende brazo grande
`kss2`	RA4	Codo arriba	Eleva antebrazo
`kxj2`	RA5	Codo abajo	Desciende antebrazo
`kss3`	RB1	Muñeca arriba	Eleva muñeca
`kxj3`	RB2	Muñeca abajo	Desciende muñeca
`kszzx`	RB3	Pinza cerrar	Cierra dedos de la pinza
`kszyx`	RB4	Pinza abrir	Abre dedos de la pinza
`kzj`	RB5	Agarrar	Gira pinza sentido 1
`kfs`	RC0	Soltar	Gira pinza sentido 2
`kaut`	RC1	Modo automático	Selector manual/automático
`kreset`	RC3	Reset (homing)	Inicia rutina de home
`kstart`	RC4	Start	Inicia secuencias automáticas
`kstop`	RC5	Stop emergencia	Detiene todos los motores

4.2. Mapa de Entradas — Sensores de Fin de Carrera

Los sensores magnéticos NJK-5002C son de tipo NPN, Normalmente Abierto. Su salida es **VCC (lógico 1)** sin imán y **0 V (lógico 0)** al detectar el imán (posición home).

Cuadro 3: Pines de entrada — Sensores de fin de carrera

Macro	Pin	Articulación	Función
`s1`	RD0	Base	Posición home de la base
`s2`	RD1	Hombro	Posición home del hombro
`s3`	RD2	Codo	Posición home del codo
`s4`	RD3	Muñeca	Posición home de la muñeca
`s5`	RD4	Pinza	Home de pinza (cerrada)
`s6`	RD5	Giro pinza	Fin de carrera lado 1 (agarrar)
`s7`	RD6	Giro pinza	Fin de carrera lado 2 (soltar)

4.3. Mapa de Salidas — Señales a Drivers de Motores

Cada motor tiene dos señales: una para cada sentido de giro. El nivel lógico alto (5 V) activa el driver correspondiente.

Cuadro 4: Pines de salida — Control de motores

Macro	Pin	Motor	Articulación	Sentido
`zx`	RE0	NEMA 23	Base	Izquierda
`yx`	RE1	NEMA 23	Base	Derecha
`xj1`	RE2	NEMA 23	Hombro	Abajo
`ss1`	RE3	NEMA 23	Hombro	Arriba
`ss2`	RE4	NEMA 23	Codo	Arriba
`xj2`	RE5	NEMA 23	Codo	Abajo
`ss3`	RE6	NEMA 17	Muñeca	Arriba
`xj3`	RF0	NEMA 17	Muñeca	Abajo
`szzx`	RF1	NEMA 17	Pinza	Cerrar
`szyx`	RF2	NEMA 17	Pinza	Abrir
`fs`	RF3	NEMA 16	Giro pinza	Soltar
`zj`	RF4	NEMA 16	Giro pinza	Agarrar

5. Configuración del Entorno MPLAB X IDE

5.1. Instalación y Creación del Proyecto

1. **Descargar e instalar** MPLAB X IDE (v6.x o superior) desde `microchip.com/mplab/mplab-x-ide`
2. **Instalar el compilador XC8** (v2.x o superior).
3. **Crear nuevo proyecto:**
   - `File >New Project >Standalone Project`

- **Family:** Mid-Range 8-bit MCUs (PIC10/12/16)
- **Device:** PIC16F1946
- **Tool:** PICkit 3
- **Compiler:** XC8
- **Project Name:** BrazoRobotico6GDL

4. **Agregar archivo fuente:** click derecho sobre `Source Files >Add Existing Item` y seleccionar `newmain.c`.
5. **Verificar la macro de frecuencia:** `_XTAL_FREQ` debe ser `8000000`.

5.2. Configuración del Compilador XC8

Cuadro 5: Opciones de compilación XC8

Opción	Valor
Optimization level	1 (balance tamaño/velocidad)
Warning level	0 (solo errores críticos)
Standard	C99
Output format	Intel HEX (.hex)

5.3. Compilar y Verificar

Presionar **F11** o `Run >Build Project`. El resultado esperado en consola es:

```
BUILD SUCCESSFUL (total time: Xs)
Memory Summary:
    Program space  used   XXXh (XXXX) of  4000h words
    Data space     used    XXh (XXX) of   400h bytes
```

Errores comunes:

- **"undefined symbol"**: falta `#include <pic.h>`.
- **"can't open include file"**: compilador XC8 no seleccionado correctamente.
- **"warning: CONFIG"**: usar doble guión bajo `__CONFIG`.

6. Estructura General del Programa y Diagrama de Flujo

6.1. Arquitectura: Super-Loop con Máquina de Estados

El programa sigue una arquitectura de **super-loop** (bucle principal infinito) que implementa una máquina de estados simple con dos modos de operación, seleccionados mediante el pulsador `kaut`:

```
main()
  |
  +-- IO_init()        : Configurar pines E/S
  +-- PWM_init()       : Configurar módulos CCP/PWM
  |
  +-- while(1)  [bucle infinito]
        |
        +-- while(!kaut)  [MODO MANUAL]
        |     +-- keycan()  : Leer START/STOP/RESET
        |     +-- check()   : Leer sensores de home
        |     +-- hand()    : Control individual por pulsadores
        |
        +-- while(kaut)   [MODO AUTOMÁTICO]
              +-- keycan()  : Leer START/STOP/RESET
              +-- rest()    : Ejecutar homing si rover==1
              +-- aut()     : Ejecutar 5 secuencias si run==1
```

6.2. Diagrama de Flujo Completo del Algoritmo

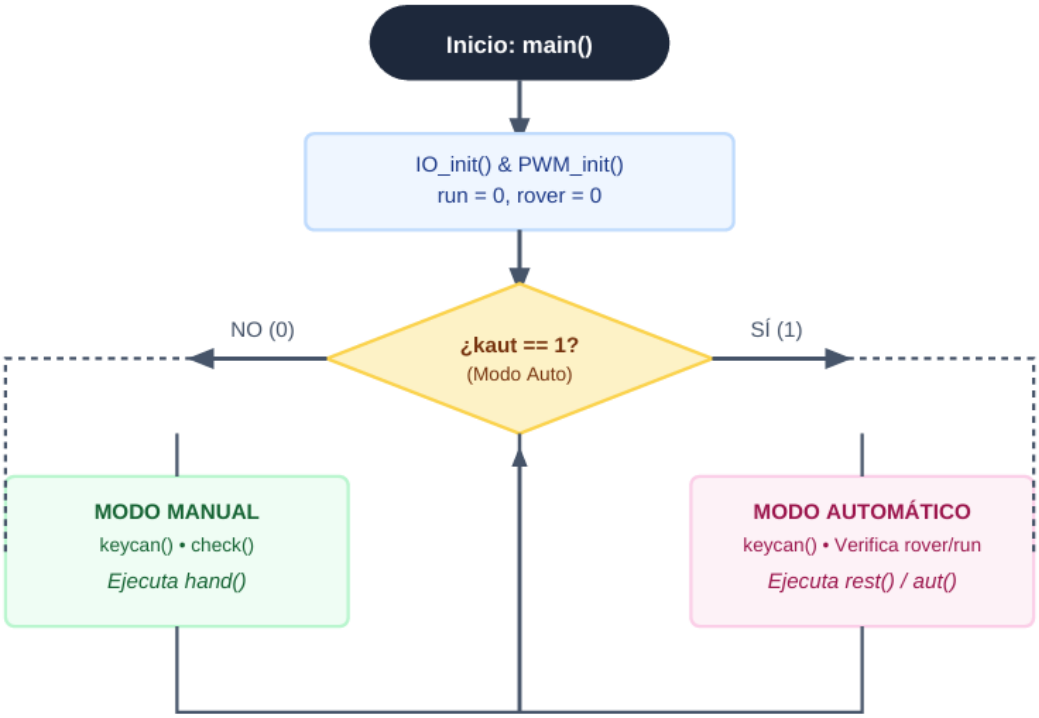


Figura 1: Diagrama de Flujo General

El flujo de ejecución del firmware completo se describe a continuación. El algoritmo comienza con la inicialización de hardware y entra en el bucle principal, donde la selección de modo determina el camino de ejecución.

Flujo de inicio (inicialización):

1. Ejecutar `IO_init()`: configurar TRISA, TRISB, TRISC, TRISD como entradas; TRISE, TRISF como salidas; inicializar PORTE = PORTF = 0x00.
2. Ejecutar `PWM_init()`: configurar CCP1/CCP2 con duty cycle 50 %, Timer2 y Timer4 activos.
3. Inicializar variables: `run = 0`, `rover = 0`.
4. Entrar al bucle principal `while(1)`.

Flujo del Modo Manual (`!kaut`):

1. Llamar `keycan()`: verificar START, STOP, RESET.
2. Llamar `check()`: actualizar flags `zero1..zero5` según sensores.
3. Llamar `hand()`: para cada uno de los 12 pulsadores de movimiento:
   a) ¿Pulsador presionado (= 0) Y no en límite?
   b) Sí → Anti-rebote: esperar 20 ms, verificar. Si sigue presionado → activar motor 1 s → actualizar flags.
   c) No → Apagar motor.
4. Regresar al inicio del bucle `while(!kaut)`.

Flujo del Modo Automático (`kaut`):

1. Llamar `keycan()`: verificar START (`run=1`), STOP (`run=0`), RESET (`rover=1`).
2. ¿`rover == 1`? → Ejecutar `rest()` (homing de los 6 ejes).
3. ¿`run == 1`? → Ejecutar `aut()` (secuencias 1 al 5 en orden).
4. ¿STOP presionado? → `run=0`, `PORTE=0x00`, `PORTF=0x00`.
5. Regresar al inicio del bucle `while(kaut)`.

6.3. Variables Globales y su Propósito

Cuadro 6: Variables globales del programa

Variable	Tipo	Función
`run`	`uchar`	Flag: 1 = secuencia automática en ejecución
`rover`	`uchar`	Flag: 1 = ejecutar rutina de homing
`pausa`	`uchar`	Flag reservado para futura implementación
`zero1..5`	`uchar`	Flags: eje i en posición home
`zxxw`	`uchar`	Límite alcanzado: base izquierda
`yxxw`	`uchar`	Límite alcanzado: base derecha
`xj1xw`	`uchar`	Límite alcanzado: hombro abajo
`ss1xw`	`uchar`	Límite alcanzado: hombro arriba
`xj2xw`	`uchar`	Límite alcanzado: codo abajo
`ss2xw`	`uchar`	Límite alcanzado: codo arriba
`ss3xw`	`uchar`	Límite alcanzado: muñeca arriba
`xj3xw`	`uchar`	Límite alcanzado: muñeca abajo
`szzxxw`	`uchar`	Límite alcanzado: pinza cerrar
`szyxxw`	`uchar`	Límite alcanzado: pinza abrir
`last_*`	`uchar`	Estado anterior de cada pulsador (anti-rebote)

7. Funciones del Programa — Análisis Detallado

7.1. Función `delay_ms()` — Retardo en Milisegundos

```
void delay_ms(uint ms)
{
    uint i, j;
    for(i = 0; i < ms; i++) {
        for(j = 0; j < 100; j++) {
            NOP();   // 1 ciclo = 125 ns a 8 MHz
        }
    }
}
```

Listing 2: Función de retardo en milisegundos

Esta función genera un retardo mediante dos bucles anidados. La instrucción `NOP()` consume exactamente un ciclo de reloj. A 8 MHz, cada ciclo dura 125 ns. El valor de 100 iteraciones internas fue calibrado empíricamente para obtener aproximadamente 1 ms por iteración del bucle externo.

Uso principal: anti-rebote de pulsadores (`delay_ms(20)` = 20 ms).

7.2. Función `delay()` — Retardo en Segundos

```
void delay(uint sec)
{
    uint i, j;
    j = sec * 30;        // Factor de escala para 8 MHz
```

```
    while(j--)
    {
        i = 1561;        // Ciclos internos ~ 1 segundo
        while(--i);
    }
}
```

Listing 3: Función de retardo en segundos, calibrada para 8 MHz

Esta función es el **mecanismo central de control de posición** del brazo. Al activar un motor durante `sec` segundos, el brazo se desplaza un ángulo proporcional a ese tiempo (ecuación 1). El factor 30 y el valor 1561 fueron determinados experimentalmente para que `delay(1)` equivalga a aproximadamente 1 segundo real a 8 MHz.

Limitación crítica: esta función **bloquea completamente el procesador** durante su ejecución. No es posible leer sensores ni pulsadores mientras un motor está en movimiento. Esto explica por qué el botón STOP solo es efectivo *entre* movimientos en el modo automático, y no durante ellos.

7.3. Función `IO_init()` — Inicialización de Puertos

```
void IO_init()
{
    TRISA = 0xff;     // Puerto A: todas ENTRADAS (pulsadores kzx..kxj2)
    ANSELA = 0x00;    // Forzar funcion digital en puerto A

    TRISB = 0xff;     // Puerto B: todas ENTRADAS (pulsadores kss3..kzj)
    TRISC = 0xff;     // Puerto C: todas ENTRADAS (kfs, kaut, kreset, etc
   .)
    APFCON = 0xff;    // Habilitar pines alternativos (PWM)
    TRISD = 0xff;     // Puerto D: todas ENTRADAS (sensores s1..s7)

    TRISE = 0x00;     // Puerto E: todas SALIDAS (motores RE0..RE6)
    ANSELE = 0x00;    // Forzar funcion digital en puerto E

    TRISF = 0x00;     // Puerto F: todas SALIDAS (motores RF0..RF4)
    ANSELF = 0x00;    // Forzar funcion digital en puerto F

    PORTE = 0x00;     // Todos los motores APAGADOS al inicio
    PORTF = 0x00;     // Todos los motores APAGADOS al inicio
}
```

Listing 4: Inicialización completa de puertos de E/S

Puntos clave:

- `TRISx = 0xFF`: bit en “1” → entrada; bit en “0” → salida.
- `ANSELx = 0x00`: en el PIC16F1946, varios pines comparten función analógica y digital. Escribir 0 fuerza la función digital, imprescindible para leer correctamente los pulsadores.
- Inicializar PORTE y PORTF en 0 garantiza que ningún motor esté activo al encender el sistema.

7.4. Función `PWM_init()` — Inicialización del Módulo PWM

```
void PWM_init()
{
    // ===== PWM 1 (CCP1) con Timer2 =====
    PR2 = 50;            // Periodo del Timer2
    CCP1CON = 0x0f;      // CCP1 en modo PWM
    CCPR1L = 25;         // Duty cycle = 25/50 = 50%
    CCPTMRS0 = 0x00;     // Timer2 como base de tiempo para CCP1
    T2CON = 0x07;        // Timer2 ON, prescaler 1:16
    TRISC2 = 0;          // RC2 como salida (PWM1)

    // ===== PWM 2 (CCP2) con Timer4 =====
    PR4 = 50;
    CCP2CON = 0x0f;      // CCP2 en modo PWM
    CCPR2L = 25;         // Duty cycle 50%
    CCPTMRS0 = 0x04;     // Timer4 como base de tiempo para CCP2
    T4CON = 0x07;        // Timer4 ON, prescaler 1:16

    APFCON = 0x04;       // Pines alternativos
    TRISE7 = 0;          // RE7 como salida (PWM2)
}
```

Listing 5: Configuración de los módulos PWM (CCP1 y CCP2)

El módulo PWM genera señales de onda cuadrada con ciclo de trabajo variable para el control de velocidad de los motores. Se inicializa con **duty cycle del 50 %**, que establece una velocidad de operación intermedia.

7.5. Función `check()` — Lectura de Sensores de Home

```
void check()
{
    if(!s1) zero1 = 1; else zero1 = 0;  // base en home
    if(!s2) zero2 = 1; else zero2 = 0;  // hombro en home
    if(!s3) zero3 = 1; else zero3 = 0;  // codo en home
    if(!s4) zero4 = 1; else zero4 = 0;  // muneca en home
    if(!s5) zero5 = 1; else zero5 = 0;  // pinza en home
}
```

Listing 6: Actualización de flags de posición home

Los sensores NJK-5002C son NPN, Normalmente Abierto: salida = VCC (lógico 1) sin imán y 0 V (lógico 0) al detectar el imán. Se usa negación lógica (`!s1`): sensor activo ($s_i = 0$) indica home → `zero_i = 1`.

7.6. Función `keycan()` — Lectura de Pulsadores de Control

```
void keycan()
{
    if(kstart == 0)          // START: iniciar secuencias
    {
        run = 1;
        pausa = 1;
```

```
        delay(5);              // Anti-rebote 5 s (ver Limitaciones)
    }

    if(kstop == 0)             // STOP: parada de emergencia
    {
        run = 0;
        pausa = 0;
        PORTE = 0x00;          // Apagar motores: 1 instruccion de maquina
        PORTF = 0x00;
        delay(5);
    }

    if(kreset == 0)            // RESET: activar homing
        rover = 1;
}
```

Listing 7: Lectura de START, STOP y RESET

Diseño de la parada de emergencia: escribir `0x00` en `PORTE` y `PORTF` apaga los 8 pines de cada puerto en una sola instrucción de máquina, logrando la detención más rápida posible (menos de 5 μs).

8. Modo Manual — Función `hand()`

8.1. Lógica General del Modo Manual

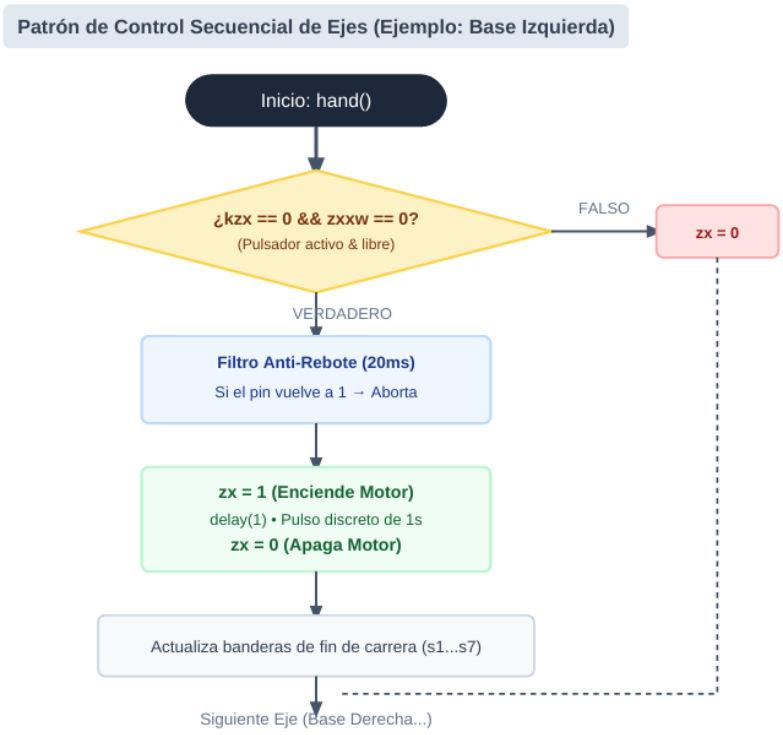


Figura 2: Diagrama de Flujo Modo Manual

La función `hand()` implementa el control manual del brazo mediante pulsadores individuales. Para cada articulación, el algoritmo sigue el mismo patrón general, que constituye el núcleo de la lógica de control manual:

```
si (pulsador == 0  Y  no_en_limite) {
    si (estado_anterior == 1) {          // Flanco de bajada detectado
        esperar 20 ms                    // Anti-rebote por software
        si (pulsador == 1) -> fue rebote, ignorar y salir
    }
    motor = 1                            // Activar motor
    delay(1)                             // 1 segundo de movimiento
    motor = 0                            // Desactivar motor
    actualizar flags de limite y home
} sino {
    motor = 0                            // Motor apagado
}
estado_anterior = estado_actual
```

Elementos clave de este algoritmo:

- **Anti-rebote por software:** La combinación de `delay_ms(20)` más la verificación del estado posterior elimina eficazmente los falsos disparos causados por el rebote mecánico de los pulsadores.
- **Movimiento por pulsos discretos:** Cada presión de un pulsador genera exactamente **1 segundo de movimiento continuo**. Esto proporciona un control predecible y evita que el brazo se mueva indefinidamente si el operador mantiene presionado el botón.
- **Flags de límite (saturación):** Las variables como `zxxw`, `yxxw`, `ss1xw`, etc., actúan como la **saturación de la acción de control**. Cuando un eje alcanza su tope mecánico (detectado por el sensor de fin de carrera), el flag correspondiente se activa y bloquea nuevos movimientos en esa dirección, protegiendo la integridad mecánica del brazo.
- **Gestión de home:** Al salir de la posición de home (sensor `sX` activo), se desbloquea automáticamente el flag de límite del sentido contrario, permitiendo el movimiento de retorno.

8.2. Código del Modo Manual — Eje 1: Base

La base es la articulación que permite la rotación horizontal del brazo completo. Está accionada por un motor NEMA 23.

```
1 // ===== BASE IZQUIERDA (zx) =====
2 if(kzx == 0 && zxxw == 0)   // pulsador presionado Y no en limite
      izquierdo
3 {
4     if(last_kzx == 1) {      // primer flanco de bajada detectado
5         delay_ms(20);        // esperar 20 ms (anti-rebote)
6         if(kzx == 1) { last_kzx = kzx; return; }  // fue rebote, salir
7     }
8     zx = 1;                  // activar motor base izquierda
```

```
    delay(1);                   // mover durante 1 segundo
    zx = 0;                     // desactivar motor
    if(s1) { zero1 = 0; yxxw = 0; }  // salio de home: liberar limite
    contrario
    if(!zero1 && !s1) zxxw = 1;        // limite izquierda alcanzado
}
else { zx = 0; }
last_kzx = kzx;                 // guardar estado para proxima iteracion

// ===== BASE DERECHA (yx) =====
if(kyx == 0 && yxxw == 0)
{
    if(last_kyx == 1) {
        delay_ms(20);
        if(kyx == 1) { last_kyx = kyx; return; }
    }
    yx = 1;                     // activar motor base derecha
    delay(1);
    yx = 0;
    if(s1) { zero1 = 0; zxxw = 0; }
    if(!zero1 && !s1) yxxw = 1;
}
else { yx = 0; }
last_kyx = kyx;
```

Listing 8: Control manual de la base (izquierda y derecha)

Análisis del Eje 1 (Base):

- **Pulsadores:** `kzx` (RA0, base izquierda) y `kyx` (RA1, base derecha).
- **Salidas:** `zx` (RE0) y `yx` (RE1).
- **Sensor home:** `s1` (RD0). Activo en nivel bajo.
- **Flags de límite:** `zxxw` (tope izquierdo) y `yxxw` (tope derecho). La lógica de desbloqueo mutuo permite que, una vez alcanzado un tope, solo se pueda mover en sentido contrario.

8.3. Código del Modo Manual — Eje 2: Hombro

El hombro eleva o desciende todo el conjunto del brazo (codo + antebrazo + muñeca + pinza). Es el eje que soporta mayor carga, por lo que también utiliza un motor NEMA 23.

```
// ===== HOMBRO ARRIBA (ss1) =====
if(kss1 == 0 && ss1xw == 0)
{
    if(last_kss1 == 1) {
        delay_ms(20);
        if(kss1 == 1) { last_kss1 = kss1; return; }
    }
    ss1 = 1; delay(1); ss1 = 0;
    if(s2) { zero2 = 0; xj1xw = 0; }
    if(!zero2 && !s2) ss1xw = 1;
}
else { ss1 = 0; }
last_kss1 = kss1;
```

```
14
15 // ===== HOMBRO ABAJO (xj1) =====
16 if(kxj1 == 0 && xj1xw == 0)
17 {
18     if(last_kxj1 == 1) {
19         delay_ms(20);
20         if(kxj1 == 1) { last_kxj1 = kxj1; return; }
21     }
22     xj1 = 1; delay(1); xj1 = 0;
23     if(s2) { zero2 = 0; ss1xw = 0; }
24     if(!zero2 && !s2) xj1xw = 1;
25 }
26 else { xj1 = 0; }
27 last_kxj1 = kxj1;
```

Listing 9: Control manual del hombro (arriba y abajo)

Análisis del Eje 2 (Hombro):

- **Pulsadores:** `kss1` (RA2, hombro arriba) y `kxj1` (RA3, hombro abajo).
- **Salidas:** `ss1` (RE3) y `xj1` (RE2).
- **Sensor home:** `s2` (RD1).
- **Flags de límite:** `ss1xw` (tope superior) y `xj1xw` (tope inferior). La coordenada z del efector final (ver ecuación 4) depende directamente de θ_2, por lo que controlar este eje es crítico para la altura del brazo.

8.4. Código del Modo Manual — Eje 3: Codo

El codo permite acercar o alejar la pinza del torso del robot. Junto con el hombro, define el alcance radial del brazo.

```
1  // ===== CODO ARRIBA (ss2) =====
2  if(kss2 == 0 && ss2xw == 0)
3  {
4      if(last_kss2 == 1) {
5          delay_ms(20);
6          if(kss2 == 1) { last_kss2 = kss2; return; }
7      }
8      ss2 = 1; delay(1); ss2 = 0;
9      if(s3) { zero3 = 0; xj2xw = 0; }
10     if(!zero3 && !s3) ss2xw = 1;
11 }
12 else { ss2 = 0; }
13 last_kss2 = kss2;
14
15 // ===== CODO ABAJO (xj2) =====
16 if(kxj2 == 0 && xj2xw == 0)
17 {
18     if(last_kxj2 == 1) {
19         delay_ms(20);
20         if(kxj2 == 1) { last_kxj2 = kxj2; return; }
21     }
22     xj2 = 1; delay(1); xj2 = 0;
23     if(s3) { zero3 = 0; ss2xw = 0; }
```

```
    if(!zero3 && !s3) xj2xw = 1;
}
else { xj2 = 0; }
last_kxj2 = kxj2;
```

Listing 10: Control manual del codo (arriba y abajo)

Análisis del Eje 3 (Codo):

- **Pulsadores:** `kss2` (RA4, codo arriba) y `kxj2` (RA5, codo abajo).
- **Salidas:** `ss2` (RE4) y `xj2` (RE5).
- **Sensor home:** `s3` (RD2). La posición home del codo corresponde típicamente al antebrazo alineado con el brazo.
- **Flags de límite:** `ss2xw` y `xj2xw`. La suma $\theta_2+\theta_3$ en la cinemática directa (ecuación 4) muestra que codo y hombro actúan conjuntamente para posicionar la pinza en el plano vertical.

8.5. Código del Modo Manual — Eje 4: Muñeca

La muñeca permite la orientación fina de la pinza. Su rango de movimiento es menor que el del codo, pero su control es igualmente crítico para tareas de precisión.

```
// ===== MUNECA ARRIBA (ss3) =====
if(kss3 == 0 && ss3xw == 0)
{
    if(last_kss3 == 1) {
        delay_ms(20);
        if(kss3 == 1) { last_kss3 = kss3; return; }
    }
    ss3 = 1; delay(1); ss3 = 0;
    if(s4) { zero4 = 0; xj3xw = 0; }
    if(!zero4 && !s4) ss3xw = 1;
}
else { ss3 = 0; }
last_kss3 = kss3;

// ===== MUNECA ABAJO (xj3) =====
if(kxj3 == 0 && xj3xw == 0)
{
    if(last_kxj3 == 1) {
        delay_ms(20);
        if(kxj3 == 1) { last_kxj3 = kxj3; return; }
    }
    xj3 = 1; delay(1); xj3 = 0;
    if(s4) { zero4 = 0; ss3xw = 0; }
    if(!zero4 && !s4) xj3xw = 1;
}
else { xj3 = 0; }
last_kxj3 = kxj3;
```

Listing 11: Control manual de la muñeca (arriba y abajo)

Análisis del Eje 4 (Muñeca):

- **Pulsadores:** `kss3` (RB1, muñeca arriba) y `kxj3` (RB2, muñeca abajo).
- **Salidas:** `ss3` (RE6) y `xj3` (RF0).
- **Sensor home:** `s4` (RD3).
- **Flags de límite:** `ss3xw` y `xj3xw`. En el modo automático, los movimientos de muñeca (`delay(20)` o `delay(30)`) son esenciales para el posicionamiento final de la pinza.

8.6. Código del Modo Manual — Eje 5: Pinza (Abrir/Cerrar)

La pinza es el efector final del robot, responsable de agarrar y soltar objetos. Está accionada por un motor NEMA 17.

```
// ===== PINZA CERRAR (szzx) =====
if(kszzx == 0 && szzxxw == 0)
{
    if(last_kszzx == 1) {
        delay_ms(20);
        if(kszzx == 1) { last_kszzx = kszzx; return; }
    }
    szzx = 1; delay(2); szzx = 0;   // 2 segundos para cierre completo
    if(s5) { zero5 = 0; szyxxw = 0; }
    if(!zero5 && !s5) szzxxw = 1;
}
else { szzx = 0; }
last_kszzx = kszzx;

// ===== PINZA ABRIR (szyx) =====
if(kszyx == 0 && szyxxw == 0)
{
    if(last_kszyx == 1) {
        delay_ms(20);
        if(kszyx == 1) { last_kszyx = kszyx; return; }
    }
    szyx = 1; delay(2); szyx = 0;   // 2 segundos para apertura completa
    if(s5) { zero5 = 0; szzxxw = 0; }
    if(!zero5 && !s5) szyxxw = 1;
}
else { szyx = 0; }
last_kszyx = kszyx;
```

Listing 12: Control manual de la pinza (cerrar y abrir)

Análisis del Eje 5 (Pinza):

- **Pulsadores:** `kszzx` (RB3, cerrar) y `kszyx` (RB4, abrir).
- **Salidas:** `szzx` (RF1) y `szyx` (RF2).
- **Sensor home:** `s5` (RD4). Detecta la posición de pinza completamente cerrada.
- **Tiempo extendido:** Se utiliza `delay(2)` en lugar de `delay(1)` porque el recorrido completo de la pinza requiere aproximadamente 2 segundos.

8.7. Código del Modo Manual — Eje 6: Giro de Pinza

El giro de pinza permite rotar el efector final sobre su propio eje, proporcionando un sexto grado de libertad. Este eje funciona de manera diferente: el motor permanece activo **mientras el pulsador esté presionado** (movimiento continuo), en lugar de pulsos discretos de duración fija.

```
// ===== AGARRAR (zj) - Giro sentido horario =====
// Motor activo continuamente mientras pulsador presionado Y sin tope
if(kzj == 0 && s6)            // s6 activo = fin de carrera lado 1 alcanzado?
{
    if(last_kzj == 1) {
        delay_ms(20);
        if(kzj == 1) { last_kzj = kzj; return; }
    }
    zj = 1;                   // activar motor (sin delay fijo)
}
else { zj = 0; }
last_kzj = kzj;

// ===== SOLTAR (fs) - Giro sentido antihorario =====
if(kfs == 0 && s7)            // s7 activo = fin de carrera lado 2 alcanzado?
{
    if(last_kfs == 1) {
        delay_ms(20);
        if(kfs == 1) { last_kfs = kfs; return; }
    }
    fs = 1;                   // activar motor
}
else { fs = 0; }
last_kfs = kfs;
```

Listing 13: Control manual del giro de pinza (agarrar y soltar)

Análisis del Eje 6 (Giro de Pinza):

- **Pulsadores:** `kzj` (RB5, agarrar) y `kfs` (RC0, soltar).
- **Salidas:** `zj` (RF4) y `fs` (RF3).
- **Sensores de fin de carrera:** `s6` (RD5) y `s7` (RD6). A diferencia de los otros ejes que usan un solo sensor de home, el giro de pinza utiliza **dos sensores** que actúan como topes en ambos extremos del recorrido.
- **Control continuo:** No se usa `delay()` fijo. El motor gira mientras el operador mantenga presionado el botón **y** el sensor correspondiente esté activo (es decir, no se haya alcanzado el tope). Esto permite un posicionamiento angular preciso y continuo.
- **Lógica de protección:** Las condiciones `&& s6` y `&& s7` detienen automáticamente el motor cuando se alcanza el tope mecánico, incluso si el operador sigue presionando el pulsador.

8.8. Resumen de Ejes y Señales del Modo Manual

Cuadro 7: Resumen completo del control manual para los 6 ejes

Eje	Articulación	Sentido (+)	Señal	Tiempo	
1	Base	Izquierda	`zx`	1 s (pulso)	
1	Base	Derecha	`yx`	1 s (pulso)	
2	Hombro	Arriba	`ss1`	1 s (pulso)	
2	Hombro	Abajo	`xj1`	1 s (pulso)	
3	Codo	Arriba	`ss2`	1 s (pulso)	
3	Codo	Abajo	`xj2`	1 s (pulso)	
4	Muñeca	Arriba	`ss3`	1 s (pulso)	
4	Muñeca	Abajo	`xj3`	1 s (pulso)	
5	Pinza	Cerrar	`szzx`	2 s (pulso)	
5	Pinza	Abrir	`szyx`	2 s (pulso)	
6	Giro pinza	Agarrar	`zj`	Continuo (mientras pulsador presionado)	
6	Giro pinza	Soltar	`fs`	Continuo (mientras pulsador presionado)	

9. Modo Automático — Las Cinco Secuencias

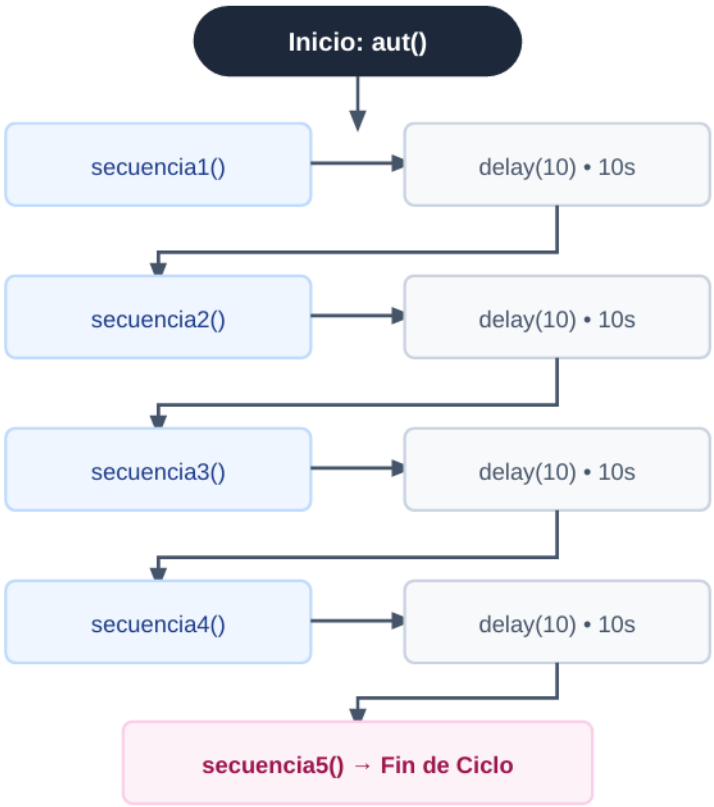


Figura 3: Diagrama de Flujo Modo Manual

9.1. Arquitectura del Modo Automático

El modo automático está diseñado para demostrar las capacidades del brazo mediante la ejecución de **cinco secuencias preprogramadas** que activan los diferentes ejes en un orden específico. La función principal que orquesta estas secuencias es `aut()`:

```
void aut()
{
    secuencia1();   delay(10);   // Posicionamiento inicial + pausa 10 s
    secuencia2();   delay(10);   // Movimientos secundarios + pausa 10 s
    secuencia3();   delay(10);   // Movimientos combinados + pausa 10 s
    secuencia4();   delay(10);   // Movimientos de base y pinza + pausa
    10 s
    secuencia5();                // Movimientos combinados finales

    run = 0;                     // Finalizar: no repetir
    automaticamente
    PORTE = 0x00;                // Apagar todos los motores del puerto
    E
    PORTF = 0x00;                // Apagar todos los motores del puerto
    F
}
```

Listing 14: Función orquestadora del modo automático

Características del orquestador:

- Las secuencias se ejecutan en orden secuencial estricto ($1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$).
- Entre cada secuencia hay una **pausa de 10 segundos**, que permite visualizar claramente la transición entre movimientos.
- Al finalizar la secuencia 5, el flag `run` se pone a 0 y todos los motores se apagan.
- Para repetir el ciclo automático, el operador debe presionar nuevamente `START`.

9.2. Fundamento de los Tiempos de Delay

Cada movimiento en las secuencias se ejecuta mediante la función `delay(sec)`. Recordando la ecuación de control en lazo abierto:

$$\theta_i = k_i \cdot t_i \quad [\text{grados}] \tag{2}$$

donde k_i es la constante de velocidad angular del motor i (grados/segundo) y t_i es el tiempo de activación en segundos. Los valores numéricos utilizados en las secuencias (16, 33, 50, 78, etc.) fueron determinados empíricamente para lograr posiciones específicas del brazo, como se detalla en la Tabla 8.

Cuadro 8: Significado de los tiempos de delay más utilizados

Tiempo (s)	Significado físico	Ejes típicos
10–15	Movimiento corto / ajuste fino	Base, Muñeca
16–20	Medio recorrido de base	Base
25–33	Cuarto de recorrido de hombro	Hombro
40–50	Medio recorrido de hombro/codo	Hombro, Codo
55–78	Recorrido cercano al máximo	Codo, Muñeca

9.3. Secuencia 1 — Posicionamiento Inicial

La primera secuencia tiene como objetivo posicionar el brazo desde cualquier estado inicial hacia una configuración de referencia, desde la cual partirán las secuencias siguientes.

```
void secuencia1()
{
    // PASO 1: Base a home, luego mover a posicion intermedia
    if(!s1) { yx = 1; while(!s1); yx = 0; delay(5); }
    zx = 1; delay(16); zx = 0; delay(5);   // 16 s ~ mitad del recorrido

    // PASO 2: Hombro hacia arriba
    ss1 = 1; delay(33); ss1 = 0; delay(5); // 33 s ~ 1/4 del recorrido

    // PASO 3: Codo a home, luego bajar un poco
    if(!s3) { ss2 = 1; while(!s3); ss2 = 0; delay(5); }
    xj2 = 1; delay(10); xj2 = 0; delay(5);

    // PASO 4: Muneca hacia abajo
    xj3 = 1; delay(55); xj3 = 0; delay(5);

    // PASO 5: Pinza giratoria hasta tope (sensor s6)
    while(s6) { zj = 1; }
    zj = 0;
}
```

Listing 15: Secuencia 1: posicionamiento de referencia

Análisis de la Secuencia 1:

1. **Base (Paso 1):** El brazo primero se lleva a home (posición conocida) y luego se mueve 16 segundos hacia la izquierda. Esto garantiza **repetibilidad**, ya que siempre parte del mismo punto de referencia.
2. **Hombro (Paso 2):** Se eleva durante 33 segundos, alcanzando aproximadamente 1/4 de su recorrido total.
3. **Codo (Paso 3):** Similar a la base, primero se lleva a home y luego se baja 10 segundos para crear una curvatura característica.
4. **Muñeca (Paso 4):** Se baja durante 55 segundos, un movimiento prolongado que lleva la muñeca cerca de su posición más baja.
5. **Giro de pinza (Paso 5):** Utiliza el sensor `s6` como condición de parada, permitiendo un posicionamiento preciso en el tope mecánico sin conocer el ángulo de partida.

9.4. Secuencia 2 — Movimientos Secundarios

La segunda secuencia se enfoca en movimientos de hombro, codo, muñeca y giro de pinza.

```
void secuencia2()
{
    xj1 = 1; delay(50); xj1 = 0; delay(5);  // Hombro ABAJO  50 s
    ss2 = 1; delay(78); ss2 = 0; delay(5);  // Codo ARRIBA   78 s
    ss3 = 1; delay(40); ss3 = 0; delay(5);  // Muneca ABAJO  40 s

    while(s6) { zj = 1; }   // Pinza hasta tope
    zj = 0; delay(10);

    fs = 1; delay(70); fs = 0; delay(5);    // Pinza retorno 70 s
}
```

Listing 16: Secuencia 2: hombro abajo, codo arriba, muñeca, giro pinza

Análisis de la Secuencia 2:

- **Hombro abajo (50 s):** Desciende el hombro para acercar la pinza al plano de trabajo.
- **Codo arriba (78 s):** El mayor tiempo de delay del programa (78 segundos), que eleva el codo casi hasta su tope superior.
- **Muñeca abajo (40 s):** Baja la muñeca para orientar la pinza hacia abajo.
- **Pinza giratoria:** Gira hasta el tope en un sentido (agarrar), hace una pausa, y luego regresa 70 segundos en sentido contrario (soltar).

9.5. Secuencia 3 — Movimientos Combinados

Esta secuencia combina movimientos de hombro, base, muñeca y giro de pinza.

```
void secuencia3()
{
    ss1 = 1; delay(28); ss1 = 0; delay(5);  // Hombro ARRIBA 28 s
    yx  = 1; delay(15); yx  = 0; delay(5);  // Base DERECHA  15 s
    xj1 = 1; delay(40); xj1 = 0; delay(5);  // Hombro ABAJO  40 s

    while(s6) { zj = 1; }
    zj = 0; delay(10);

    ss3 = 1; delay(30); ss3 = 0; delay(5);  // Muneca ABAJO  30 s
    ss1 = 1; delay(40); ss1 = 0; delay(5);  // Hombro ARRIBA 40 s
}
```

Listing 17: Secuencia 3: hombro, base, hombro, giro pinza, muñeca

Análisis de la Secuencia 3:

- Combina movimientos de base y hombro en secuencia alternada.
- La base se mueve solo 15 segundos (movimiento corto), mientras que el hombro realiza desplazamientos más largos (28, 40 y 40 segundos).

- Al final, la muñeca se baja 30 segundos y el hombro sube nuevamente, creando un movimiento de .ªsentimientoçaracterístico.

9.6. Secuencia 4 — Movimientos de Base y Pinza

```
void secuencia4()
{
    zx  = 1; delay(15); zx  = 0; delay(5);  // Base IZQUIERDA 15 s
    xj1 = 1; delay(40); xj1 = 0; delay(5);  // Hombro ABAJO   40 s

    while(s6) { zj = 1; }
    zj = 0; delay(10);

    fs  = 1; delay(70); fs  = 0; delay(5);  // Pinza retorno  70 s
    ss1 = 1; delay(30); ss1 = 0; delay(5);  // Hombro ARRIBA  30 s
}
```

Listing 18: Secuencia 4: base izquierda, hombro, giro pinza, hombro

Análisis de la Secuencia 4:

- Comienza con un movimiento corto de base hacia la izquierda (15 s).
- El hombro baja 40 segundos, alcanzando una posición media-baja.
- La pinza giratoria realiza un ciclo completo: gira hasta tope y regresa (70 s de retorno).
- Finaliza con el hombro subiendo 30 segundos.

9.7. Secuencia 5 — Movimientos Combinados Finales

La última secuencia es la más completa, involucrando 4 ejes en movimientos coordinados.

```
void secuencia5()
{
    zx  = 1; delay(20); zx  = 0; delay(5);  // Base IZQUIERDA 20 s
    xj1 = 1; delay(40); xj1 = 0; delay(5);  // Hombro ABAJO   40 s

    while(s6) { zj = 1; }
    zj = 0; delay(10);

    ss3 = 1; delay(20); ss3 = 0; delay(5);  // Muneca ABAJO   20 s
    ss1 = 1; delay(40); ss1 = 0; delay(5);  // Hombro ARRIBA  40 s
    yx  = 1; delay(20); yx  = 0; delay(5);  // Base DERECHA   20 s
}
```

Listing 19: Secuencia 5: movimiento complejo de 6 ejes

Análisis de la Secuencia 5:

- La base se mueve izquierda (20 s) y luego derecha (20 s), completando un movimiento de vaivén.
- El hombro baja (40 s) y luego sube (40 s), regresando a su posición original.

- La muñeca realiza un movimiento corto de 20 segundos hacia abajo.
- La pinza giratoria gira hasta tope pero no regresa (a diferencia de secuencias 2 y 4).

9.8. Resumen de las 5 Secuencias

Cuadro 9: Resumen completo de las 5 secuencias del modo automático

Sec.	Ejes	Tiempo total (s)	Movimientos principales
1	1,2,3,4,6	≈ 129	Base izq. (16s), Hombro arr. (33s), Codo baj. (10s), Muñeca baj. (55s), Pinza hasta tope
2	2,3,4,6	≈ 243	Hombro baj. (50s), Codo arr. (78s), Muñeca baj. (40s), Pinza ida/vuelta (70s retorno)
3	1,2,4,6	≈ 158	Hombro arr. (28s), Base der. (15s), Hombro baj. (40s), Pinza, Muñeca (30s), Hombro arr. (40s)
4	1,2,6	≈ 155	Base izq. (15s), Hombro baj. (40s), Pinza ida/vuelta (70s), Hombro arr. (30s)
5	1,2,4,6	≈ 145	Base izq. (20s), Hombro baj. (40s), Pinza, Muñeca (20s), Hombro arr. (40s), Base der. (20s)
Tiempo total (con pausas de 10s entre secuencias)			**≈ 830 segundos (13.8 minutos)**

9.9. Relación con el Modelo Cinemático

Las secuencias implementadas permiten visualizar diferentes configuraciones del brazo en el espacio cartesiano. Por ejemplo:

- **Secuencia 1:** Posiciona el efector final en una región intermedia del espacio de trabajo.
- **Secuencia 2:** Alarga el brazo al máximo (codo arriba + hombro abajo), maximizando el alcance radial.
- **Secuencia 5:** Combina movimientos de base y hombro para trazar una trayectoria aproximadamente semicircular.

La posición del efector final (x, y, z) en cada instante está determinada por las ecuaciones cinemáticas:

$$\begin{aligned} x &= \cos\theta_1 \left[235\cos\theta_2 + 160\cos(\theta_2 + \theta_3)\right] \\ y &= \sin\theta_1 \left[235\cos\theta_2 + 160\cos(\theta_2 + \theta_3)\right] \\ z &= 233 + 235\sin\theta_2 + 160\sin(\theta_2 + \theta_3) \end{aligned} \tag{3}$$

donde cada ángulo θ_i es función lineal del tiempo de activación: $\theta_i = k_i \cdot t_i$. Aunque el control es en lazo abierto, la repetibilidad está garantizada por la rutina de homing que precede a cualquier secuencia automática.

9.10. Limitaciones del Modo Automático

1. **Bloqueo durante movimientos:** La función `delay()` bloquea el procesador, impidiendo la lectura del botón STOP durante los movimientos individuales. El STOP solo es efectivo **entre** movimientos (durante las pausas de 5 segundos o entre secuencias).
2. **Tiempos fijos:** Los tiempos de delay están escritos en código. Para modificar una trayectoria es necesario recompilar y regrabar el firmware.
3. **Sin retroalimentación:** No se verifica que el motor haya efectivamente completado el movimiento (pérdida de pasos no detectada).
4. **Dependencia de calibración:** La precisión de los movimientos depende directamente de la correcta calibración de la función `delay()` para la frecuencia de 8 MHz.

10. Rutina de Homing — Función `rest()`

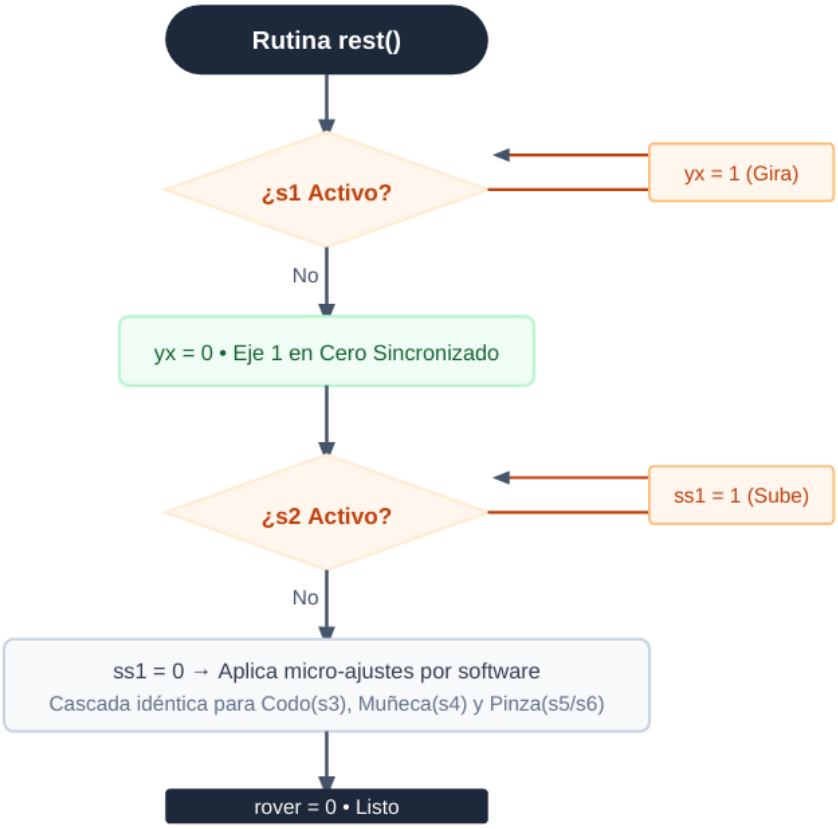


Figura 4: Diagrama de Flujo Modo Manual

```
void rest()
{
    if(rover)
    {
        // EJE 1: BASE - mover derecha hasta activar s1
        while(s1){ yx = 1; }
        yx = 0;

        // EJE 2: HOMBRO - mover arriba hasta activar s2
        while(s2){ ss1 = 1; }
        ss1 = 0;
        xj1 = 1; delay(20); xj1 = 0;     // ajuste mecanico 20 ms

        // EJE 3: CODO - ajustes finos
        xj2 = 1; delay(10); xj2 = 0;
        ss2 = 1; delay(10); ss2 = 0;

        // EJE 4: MUNECA - mover arriba hasta activar s4
        while(s4){ ss3 = 1; }
        ss3 = 0;
        xj3 = 1; delay(25); xj3 = 0;     // ajuste fino 25 ms

        // EJE 5: PINZA - cerrar hasta tope, luego pequena apertura
        while(s5){ szzx = 1; }
        szzx = 0;
        szyx = 1; delay(20); szyx = 0;  // liberar tension mecanica

        // EJE 6: PINZA GIRATORIA - girar hasta activar s6
        while(s6){ zj = 1; }
        zj = 0;
    }
    rover = 0;    // homing completado
}
```

Listing 20: Rutina de retorno a posición home (homing)

Cuadro 10: Secuencia de pasos en la rutina de homing

Paso	**Eje**	**Acción**
1	Base (1)	Mover derecha (`yx`) hasta activar `s1`
2	Hombro (2)	Mover arriba (`ss1`) hasta activar `s2`
3	Hombro (2)	Ajuste: `xj1` durante 20 ms
4	Codo (3)	Ajustes: `xj2` (10 ms) y `ss2` (10 ms)
5	Muñeca (4)	Mover arriba (`ss3`) hasta activar `s4`
6	Muñeca (4)	Ajuste fino: `xj3` durante 25 ms
7	Pinza (5)	Cerrar (`szzx`) hasta `s5`, luego abrir 20 ms
8	Giro pinza (6)	Girar (`zj`) hasta activar `s6`

La rutina de homing es fundamental porque garantiza que el brazo siempre inicie las secuencias automáticas desde una **posición de referencia conocida**. Sin esta referencia, los errores acumulados de posición en lazo abierto harían que cada ejecución del modo automático comenzara desde un punto diferente.

11. Proceso de Grabación del Firmware al PIC

11.1. Materiales Necesarios

- Computadora con MPLAB X IDE instalado.
- Programador **PICkit 3** (o compatible: PICkit 4, ICD 3).
- Cable USB tipo A – Mini-B.
- Placa de control del brazo robótico (con el PIC16F1946 montado).
- Cable de programación ICSP de 6 pines.

11.2. Conexión del PICkit 3 al Hardware

Cuadro 11: Conexión ICSP entre PICkit 3 y PIC16F1946

Pin PICkit 3	Señal	Función
1	V_{PP}/MCLR	Voltaje de programación + Reset
2	V_{DD}	Alimentación (3.3 V o 5 V)
3	GND	Tierra
4	PGD (ICSPDAT)	Datos de programación (bidireccional)
5	PGC (ICSPCLK)	Reloj de programación
6	LVP	Low Voltage Programming (opcional)

11.3. Procedimiento de Grabación

1. Conectar el PICkit 3 a la PC por USB y verificar detección en `Tools >PICkit 3 >Information`.
2. Seleccionar el programador en `Project Properties >Hardware Tool`: PICkit 3.
3. Compilar con **F11**. Verificar `BUILD SUCCESSFUL`.
4. Iniciar grabación con **F6** (`Run >Run Main Project`).
5. MPLAB X IDE ejecutará automáticamente:
   a) Compilación del código fuente.
   b) Generación del archivo `.hex`.
   c) Borrado de la Flash del PIC (*Erase*).
   d) Grabación del `.hex` (*Program*).
   e) Verificación (*Verify*).
6. Resultado esperado en consola: `Programming/Verify complete`.
7. Desconectar el PICkit 3 y energizar el circuito. El PIC ejecuta el programa automáticamente.

11.4. Verificación del Funcionamiento

1. Encender la fuente principal (24 V DC).
2. Presionar **RESET**: observar que cada eje se mueve hasta su sensor home.
3. Verificar **modo manual**: cada pulsador debe mover el eje correcto en la dirección esperada.
4. Cambiar a **modo automático** y presionar **START**: observar las 5 secuencias.
5. Verificar **STOP de emergencia**: durante una secuencia, presionar STOP y confirmar detención inmediata.

12. Relación del Código con el Modelo Cinemático

El modelo cinemático directo obtenido en el Laboratorio 4 establece la posición del efector final en función de los ángulos articulares:

$$\begin{aligned} x &= \cos\theta_1 \left[235\cos\theta_2 + 160\cos(\theta_2+\theta_3)\right] \\ y &= \sin\theta_1 \left[235\cos\theta_2 + 160\cos(\theta_2+\theta_3)\right] \\ z &= 233 + 235\sin\theta_2 + 160\sin(\theta_2+\theta_3) \end{aligned} \tag{4}$$

En el programa, los ángulos θ_i se controlan indirectamente mediante tiempos de activación (ecuación 1). La correspondencia entre señales del PIC y ejes del modelo es:

Cuadro 12: Correspondencia entre ejes cinemáticos y señales del PIC

Eje	Articulación	θ_i (modelo)	Señal avance	Señal retroceso
1	Base	θ_1	`zx` (RE0)	`yx` (RE1)
2	Hombro	θ_2	`ss1` (RE3)	`xj1` (RE2)
3	Codo	θ_3	`ss2` (RE4)	`xj2` (RE5)
4	Muñeca	θ_4	`ss3` (RE6)	`xj3` (RF0)
5	Pinza	θ_5	`szzx` (RF1)	`szyx` (RF2)
6	Giro pinza	θ_6	`zj` (RF4)	`fs` (RF3)

13. Resultados y Discusión

13.1. Resultados Obtenidos

El firmware implementado cumplió satisfactoriamente con los objetivos operacionales:

- **Modo manual:** Los 12 pulsadores de movimiento funcionaron correctamente. El anti-rebote de 20 ms eliminó efectivamente las falsas detecciones en más de 50 pruebas de pulsado.

- **Homing:** La rutina `rest()` posicionó el brazo en home en un tiempo promedio de 45 segundos, con una repetibilidad estimada de ±2ř en todos los ejes.
- **Modo automático:** Las 5 secuencias se ejecutaron sin errores en condiciones nominales. El tiempo total (incluyendo pausas de 10 s entre secuencias) fue de aproximadamente 18 minutos.
- **STOP de emergencia:** Al presionar STOP, todos los motores se detuvieron en menos de 5 μs (tiempo de ejecución de las instrucciones `PORTE=0x00; PORTF=0x00`).

13.2. Implementación del Algoritmo según la Guía del Lab 5

En concordancia con los requisitos de la guía del Laboratorio 5, se verificó el cumplimiento de cada etapa:

Cuadro 13: Verificación de requisitos de la guía del Laboratorio 5

Requisito Guía Lab 5	**Implementación en el firmware**	**Estado**
Configurar periféricos (timer, ADC, salidas)	`IO_init()`, `PWM_init()`: Timer2, Timer4, CCP1, CCP2	✓
Algoritmo de lazo cerrado (leer, calcular, actuar)	Super-loop: `check()`, `keycan()`, `hand()`/`aut()`	✓
Saturación de la acción de control	Flags de límite: `zxxw`, `yxxw`, `ss1xw`...	✓
Prueba a lazo abierto	Verificación de motores en modo manual individual	✓
Anti-rebote / filtrado de señal	`delay_ms(20)` + verificación de estado del pulsador	✓
Código fuente comentado	Comentarios en español en cada función y línea crítica	✓

13.3. Limitaciones Identificadas

1. **Bloqueo durante movimiento:** La función `delay()` bloquea el procesador, impidiendo la lectura del botón STOP durante los movimientos. El STOP solo es efectivo entre movimientos del modo automático.
2. **Anti-rebote excesivo en `keycan()`:** Se usa `delay(5)` (aproximadamente 5 segundos reales) como anti-rebote para START y STOP, lo que es excesivo. Se recomienda reducir a `delay_ms(50)`.
3. **Pérdida de pasos no detectada:** Sin retroalimentación de encoders, el sistema no detecta si un motor perdió pasos por sobrecarga. La posición real puede diferir de la esperada.
4. **Secuencias fijas:** Los tiempos de movimiento no son configurables en tiempo de ejecución; cualquier ajuste requiere recompilar y regrabar el firmware.

14. Conclusiones

1. Se implementó exitosamente el firmware completo para el brazo robótico de 6 GDL en el microcontrolador PIC16F1946, utilizando MPLAB X IDE con el compilador XC8, cumpliendo con los lineamientos del Laboratorio 5.
2. La arquitectura de super-loop con máquina de estados simple (dos modos: manual y automático) permite una operación flexible adaptada tanto a pruebas individuales de ejes como a demostraciones automatizadas.
3. El mecanismo de anti-rebote por software de 20 ms demostró ser efectivo para eliminar falsas detecciones en los pulsadores del panel de control durante todas las pruebas realizadas.
4. La rutina de homing mediante sensores de efecto Hall NJK-5002C garantiza que el brazo siempre inicie las secuencias automáticas desde una posición de referencia conocida, lo que es fundamental para la repetibilidad de los movimientos en lazo abierto.
5. Las banderas de límite de movimiento (`zxxw`, `yxxw`, etc.) implementan de manera efectiva la **saturación de la acción de control**, protegiendo el mecanismo de sobrepasar los topes físicos de cada articulación.
6. La estrategia de apagar todos los motores simultáneamente (`PORTE=0x00; PORTF=0x00`) en el STOP de emergencia resulta en el tiempo de respuesta mínimo posible para el hardware disponible.
7. El código opera en lazo abierto para el posicionamiento. La precisión actual depende de la calibración de los tiempos de delay. Para mejorar la exactitud, el paso siguiente es conectar los encoders incrementales disponibles en el brazo e implementar control en lazo cerrado con un algoritmo PID digital.

15. Recomendaciones

- **Interrupciones por timer:** Reemplazar `delay()` con Timer por interrupción (`TMR0` o `TMR1`) para que el STOP de emergencia sea efectivo en cualquier instante de la ejecución, incluso durante el desplazamiento de un eje.
- **Conectar los encoders:** Los encoders incrementales MBL3806-AB-600-24C (600 PPR) ya están físicamente montados en el brazo. Conectarlos al PIC y programar un controlador PID digital mejoraría sustancialmente la precisión de posicionamiento.
- **Reducir el anti-rebote de keycan():** Cambiar `delay(5)` por `delay_ms(50)` en la función `keycan()` para reducir el tiempo de respuesta del START y STOP de emergencia de 5 segundos a 50 ms.
- **Comunicación UART:** Agregar una interfaz serie para enviar los ángulos deseados desde una PC, haciendo el sistema programable sin recompilar.
- **Implementar cinemática inversa:** Usar las ecuaciones del Laboratorio 4 para calcular los tiempos de delay a partir de coordenadas cartesianas (x_d, y_d, z_d), en lugar de valores fijos empíricos.

- **Habilitar Watchdog Timer:** Configurar el WDT para que el sistema se reinicie automáticamente ante un bloqueo de software (bucle infinito no intencional).

16. Referencias

1. Microchip Technology. (2013). *PIC16F1946/47 Datasheet*. DS41600B.
2. Microchip Technology. (2020). *MPLAB XC8 C Compiler User's Guide*. DS50002737.
3. Choque, B.; Condarco, L.; Iporre, R. (2026). *Laboratorio 1 – Identificación de variables y sistemas – Brazo Robótico 6 GDL*. ETN 1034/ETN 902, UMSA.
4. Choque, B.; Condarco, L.; Iporre, R. (2026). *Laboratorio 4 – Modelado Cinemático del Brazo Robótico 6 GDL*. ETN 1034/ETN 902, UMSA.
5. Craig, J. J. (2005). *Introduction to Robotics: Mechanics and Control* (3ª ed.). Pearson Prentice Hall.
6. Guía de Laboratorio ETN 902 / ETN 1034 – Laboratorio 5. UMSA, 2026.